

Retail Vision Intelligence System

Trabalho Prático #2 - LIACD 2025/2026

Inês Santos

Licenciatura em Inteligência Artificial e Ciência de Dados

Universidade da Beira Interior

Covilhã, Portugal

ines.matos.santos@ubi.pt

Abstract—Este trabalho apresenta o *Retail Vision Intelligence System*, um sistema de inspeção contínua de prateleiras de retalho que integra análise visual multimodal com memória histórica e geração automática de alertas e relatórios. O sistema utiliza o modelo Gemini 2.5 Flash da Google como motor de visão computacional, ChromaDB como *vector store* persistente para memória RAG, e um motor de regras baseado em linguagem natural. São comparadas três estratégias de *prompting*, *zero-shot*, *chain-of-thought* e *few-shot*, sobre um conjunto de 15 imagens com *ground truth* anotado. Os resultados mostram que a estratégia CoT elimina completamente as alucinações (0,0%) ao custo de menor precisão na classificação de severidade, enquanto a estratégia *zero-shot* apresenta o maior *suggestion bias* (42,9% de alucinações). O sistema completo integra cinco componentes independentes orquestrados por uma interface CLI.

Index Terms—visão computacional, LLM multimodal, RAG, inspeção de prateleiras, *prompting*, retalho inteligente

I. INTRODUÇÃO E ARQUITETURA

O *Retail Vision Intelligence System* resolve um problema operacional concreto no retalho: a invisibilidade das falhas de restock. Uma zona pode ter afluência elevada e *dwell time* longo, mas se um produto estiver mal posicionado ou uma prateleira vazia, a loja perde vendas de forma silenciosa para qualquer sistema baseado apenas em deteção de presença. Este trabalho constrói a camada de inteligência visual que falta ao sistema do Projeto 1, adicionando análise de imagens, memória histórica indexada e alertas configurados em linguagem natural pelo gestor de loja.

A. Arquitetura do Sistema

A arquitetura do sistema foi desenhada seguindo princípios de modularidade e desacoplamento de componentes, garantindo que a falha de um módulo não comprometa a integridade dos restantes. O fluxo de dados segue um *pipeline* sequencial, conforme ilustrado na Fig. 1, que permite uma orquestração centralizada através da interface CLI:

- 1) **ShelfInspector** (`shelf_inspector.py`): O núcleo de visão computacional. Responsável pela tradução da entrada visual (imagem) em conhecimento estruturado (JSON), extraíndo *issues*, *fill rate* e justificações textuais.

- 2) **RuleEngine** (`rule_engine.py`): Motor de decisão que traduz regras de negócio em linguagem natural para predicados lógicos, validando os resultados das inspeções contra as normas da loja.
- 3) **RagMemory** (`rag_memory.py`): Sistema de armazenamento vetorial que indexa inspeções passadas no ChromaDB, permitindo consultas semânticas e análise temporal.
- 4) **ReportGenerator** (`report_generator.py`): Módulo de síntese que gera relatórios executivos em formato Markdown, estruturados para leitura humana.
- 5) **RetailInterface** (`interface.py`): Interface de orquestração que gere o ciclo de vida da execução e a interação com o utilizador final.

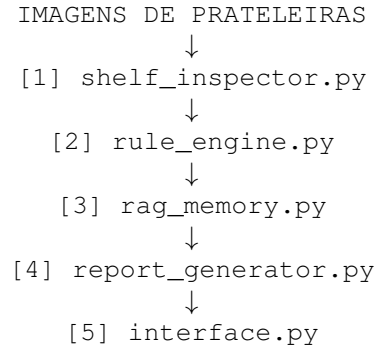


Fig. 1. Diagrama de arquitetura da *pipeline* do sistema.

B. Decisões de Design

1) **Evolução do Modelo de Inferência:** A seleção do motor de inferência foi um processo iterativo e de diagnóstico técnico rigoroso. Inicialmente, o sistema foi desenhado para utilizar a versão 1.5 Flash do modelo Gemini, devido à sua eficiência prometida. Contudo, durante a fase de integração com o *pipeline* de dados local, verificaram-se instabilidades críticas no *handshake* da API e inconsistências recorrentes na serialização do formato JSON solicitado. Após testes exaustivos, optou-se pela migração para o modelo **Gemini 2.5 Flash**. Esta versão demonstrou uma superior estabilidade na conexão e uma aderência muito mais estrita ao *schema* JSON definido no *system prompt*, sendo fundamental para

a resiliência do sistema. A temperatura foi fixada a 0,0 em todos os componentes para garantir a reprodutibilidade dos resultados.

2) *Cache, Persistência e Robustez*: A estratégia de *caching* foi implementada através de *hashing* MD5, utilizando como chave a combinação da imagem de entrada e do *prompt* utilizado. Esta abordagem é essencial num ambiente de desenvolvimento com quotas limitadas (1500 req/dia), evitando o reprocessamento de imagens e garantindo eficiência de custos.

Quanto ao armazenamento, o ChromaDB com `PersistentClient` foi a tecnologia escolhida por permitir uma arquitetura *serverless*, operando localmente sem a necessidade de *overhead* de infraestrutura. Os *embeddings* são gerados de forma automática via *sentence-transformers*, garantindo que a memória histórica do sistema tenha riqueza semântica suficiente para consultas complexas.

Finalmente, a gestão de erros foi desenhada para a continuidade de serviço. A interface nunca expõe *stack traces* ao utilizador. Todos os componentes incorporam blocos de tratamento de exceções (*try-except*) que garantem que, mesmo em caso de falha da API ou esgotamento de quota, o sistema retorne resultados parciais baseados em dados de *cache*, mantendo a operabilidade básica do sistema.

II. DATASET E ESTRATÉGIA DE AQUISIÇÃO

A robustez de um modelo multimodal no retalho depende da heterogeneidade dos dados de treino e teste. O *dataset* de avaliação utilizado neste trabalho não se restringiu a uma única fonte, tendo sido adotada uma estratégia de curadoria agregada. Para além do dataset de referência SKU-110K, foram integradas amostras de múltiplos repositórios públicos para garantir a diversidade de contextos (layouts de prateleiras, tipos de iluminação e categorias de produtos).

A. Origem dos Dados

A estratégia de aquisição baseou-se na fusão de cinco fontes distintas, visando cobrir casos de uso que datasets genéricos frequentemente omitem:

- **SKU-110K e Grocery-Planogram-Control-Dataset**: Utilizados para garantir representatividade em cenas de elevada densidade de produtos e layouts de prateleiras típicos de supermercados.
- **Grocery Store Dataset (Kaggle) e Retail Store Images**: Integrados para fornecer variação em produtos de conveniência e escalas de prateleiras menores.
- **Supermarket Shelves Dataset (Roboflow)**: Essencial para validar cenários com códigos de barras e etiquetas de preço variáveis.
- **Captura Própria**: Foram incluídas fotografias próprias para colmatar a ausência de cenários específicos de "ruído real", como reflexos em divisórias de plástico, iluminação

variável e obstruções parciais que testam o limite da capacidade de raciocínio do modelo.

Esta diversidade permite uma avaliação mais rigorosa, garantindo que o modelo não sofra de sobre-ajuste (*overfitting*) a um único estilo de prateleira ou padrão de iluminação.

B. Distribuição e Cenários

O *dataset* final foi compilado manualmente, compreendendo 15 imagens anotadas com *ground truth* rigoroso, cobrindo cinco categorias críticas de anomalias operacionais.

TABLE I
DISTRIBUIÇÃO DAS 15 IMAGENS POR CATEGORIA DE TESTE

Tipo de Cenário	Sufixo	N
Prateleira Normal	(n)	3
Prateleira Vazia (parcial/total)	(v)	3
Violação de planograma	(vio)	3
Prateleira Desordenada/Suja	(s)	3
Caso ambíguo	(a)	3
Total		15

C. Limitações Técnicas

A análise de dados revelou limitações intrínsecas ao processo de captura: as três imagens da categoria “ambíguo” apresentam ruído digital e blocos pretos de privacidade que obscurecem zonas da prateleira. Embora estas falhas dificultem a deteção por parte de qualquer sistema, elas servem como um teste de robustez, avaliando como o modelo lida com a incerteza visual.

Adicionalmente, o *ground truth* foi definido com um único *issue* primário por imagem. Esta simplificação, embora necessária para a métrica de *recall*, favorece artificialmente sistemas que priorizam apenas a anomalia mais óbvia, podendo penalizar sistemas que detetam múltiplos problemas granulares de forma correta (falsos negativos no *ground truth*).

III. ANÁLISE VISUAL

O componente `shelf_inspector.py` constitui o motor de visão do sistema, utilizando o Gemini 2.5 Flash como *backbone* de processamento. A interação com o modelo é realizada através de uma pipeline onde a imagem é carregada via `client.files.upload()` e submetida com um *prompt* estruturado, forçando a saída em formato JSON nativo com `temperature=0.0` para garantir determinismo nas respostas.

A. Estratégias de Prompting

Para avaliar o desempenho do modelo, foram implementadas três estratégias de *prompting* com níveis crescentes de complexidade cognitiva:

- 1) **Estratégia A (Zero-Shot Direto):** O modelo assume o papel de um "auditor de inventário extremamente minucioso e pessimista". O *prompt* contém instruções diretas sobre tipos de problemas e regras de classificação de *status*. Esta estratégia serve como *baseline* para avaliar a tendência de *bias* do modelo.
- 2) **Estratégia B (Chain-of-Thought Visual):** Estrutura a inspeção em três passos obrigatórios: Varrimento Global, Análise por Zonas (esquerda para a direita) e Classificação com justificação. O raciocínio é obrigatoriamente registado no campo `model_reasoning` antes da tomada de decisão, visando reduzir alucinações via auto-verificação.
- 3) **Estratégia C (Few-Shot):** Incorpora exemplos textuais de inspeções anteriores (uma prateleira vazia crítica e um produto tombado) no próprio *prompt*. Como não é viável passar múltiplas imagens na mesma chamada de API sem elevar drasticamente o consumo de *tokens*, optei por descrições textuais ricas que servem como "guia de estilo" para o *output*.

B. Resultados Quantitativos

A Tabela II resume a performance das três abordagens sobre o *dataset* de 15 imagens.

TABLE II
COMPARAÇÃO DAS TRÊS ESTRATÉGIAS DE *prompting*

Métrica	Estratégia A	Estratégia B	Estratégia C
JSON Parse Rate	93,3%	93,3%	100,0%
Hallucination Rate	42,9%	0,0%	6,7%
Issue Detection Rate	36,4%	36,4%	25,0%
False Positive Rate	93,3%	69,2%	80,0%
Severity Accuracy	50,0%	25,0%	33,3%

C. Análise Crítica dos Resultados

Robustez do Formato (JSON Parse Rate). A Estratégia C atingiu a perfeição (100%), validando que o *Few-shot* é a técnica mais eficaz para forçar a adesão ao *schema* JSON esperado. As falhas residuais nas Estratégias A e B (93,3%) não foram erros de estrutura do modelo, mas sim interrupções por erros 503 da API (*Service Unavailable*), que foram geridos via persistência em cache.

Mitigação de Alucinações (Hallucination Rate). O resultado mais notável é a performance da Estratégia B (0,0% de alucinações). Ao obrigar o modelo a descrever o cenário no campo `model_reasoning` antes de inferir um problema, a Estratégia B forçou o LMM a realizar uma verificação visual antes da classificação. Em contraste, a Estratégia A apresentou uma elevada taxa de alucinações (42,9%), impulsionada por um *suggestion bias*: ao sugerir problemas específicos (ex: "produtos da Air Wick") no *prompt*, o modelo "inventou" a presença desses produtos onde eles não existiam.

Deteção e Precisão. A *Issue Detection Rate* (recall) e a *False Positive Rate* (FPR) apresentam um compromisso clássico. Embora o *recall* pareça baixo (25–36%), deve notar-se que o nosso *ground truth* é conservador (apenas 1 *issue* anotado por imagem). O modelo, ao ser minucioso, detetou frequentemente múltiplos problemas válidos que não estavam marcados no *ground truth*, resultando numa FPR artificialmente alta.

Casos de Falha Relevantes.

- **Suggestion Bias (Exemplo 1):** Na Estratégia A, o modelo reportou desalinhamentos em prateleiras de Sprite perfeitamente arrumadas. O *prompt* era demasiado específico sobre "problemas esperados", fazendo com que o modelo priorizasse a confirmação das instruções do utilizador em detrimento da evidência visual.
- **Limitações da CoT (Exemplo 10):** A Estratégia B, embora lógica, falhou em classificar a severidade corretamente. O modelo focou-se intensivamente em verificar se o *fill rate* estava correto, mas ignorou a posição relativa dos produtos, demonstrando que o *Chain-of-Thought* guia o raciocínio, mas não garante a compreensão de violações de planograma mais subtis.

IV. RULE ENGINE

O `rule_engine.py` atua como a camada de abstração entre a intenção de negócio do gestor e os dados técnicos produzidos pelo sistema. A sua função primordial é a tradução de linguagem natural para uma representação estruturada (JSON), permitindo que utilizadores sem conhecimentos de programação definam políticas de gestão de stock complexas de forma intuitiva.

A. Arquitetura da Validação Semântica

A engine não se limita a uma conversão literal; ela incorpora um protocolo de validação semântica executado pelo Gemini 2.5 Flash. Ao receber uma regra, o modelo avalia a completude da instrução com base em diretrizes rígidas, garantindo que o sistema apenas execute filtros que possuam parâmetros matematicamente definidos. Esta abordagem minimiza o risco de falsos negativos por ambiguidade, assegurando que o motor de execução determinístico trabalhe apenas com dados validados.

B. Estratégia de Deteção de Ambiguidades

O sistema opera sob duas diretrizes estritas de processamento:

- 1) **Diretriz 1 (Ambiguidade Crítica):** Caso a instrução do gestor omita parâmetros vitais (ex: zona de atuação, limiar numérico de "*fill rate*"), o sistema marca o campo `is_valid` como `false`. O modelo não tenta adivinhar intenções, pelo contrário, devolve uma lista detalhada

das lacunas detetadas, forçando uma interação de clarificação. Isto garante que o motor de execução nunca opere sobre premissas falsas ou assumidas.

- 2) **Diretriz 2 (Valores por Defeito):** Para instruções semanticamente claras mas tecnicamente incompletas (ex: omissão de nível de severidade), o sistema assume um comportamento pró-ativo, preenchendo os campos com valores de consenso de indústria (defaults) e registrando essas suposições no campo `assumptions` do JSON, mantendo o utilizador informado.

C. Exemplos de Conversão

A Tabela III ilustra a eficácia do sistema na triagem de instruções de negócio, demonstrando a capacidade de distinguir entre intenções completas e vagas.

TABLE III
EXEMPLOS DE CONVERSÃO E VALIDAÇÃO DE REGRAS

Regra do Gestor	Válida?
"Avisa-me se a prateleira estiver vazia."	Não - falta zona e limiar
"Avisa quando a prateleira inferior da zona Z_S2 estiver > 30% vazia."	Sim - todos os parâmetros presentes
"Quando o fill rate cair abaixo de 60% entre as 10h e as 13h."	Não - falta zona específica
"Na Z_S1, se não houver laticínios, é crítico."	Sim - zona e nível explícitos

D. Lógica de Execução Determinística

Uma vez validado o objeto JSON, o executor `execute_rules` aplica a lógica de filtragem de forma determinística sobre o `output` do `ShelfInspector`. O processo verifica três condições sequenciais: (1) a `zone_id` da inspeção coincide com o `zone_filter`; (2) o `issue` detetado pertence aos `issue_types` monitorizados; (3) o `fill rate` medido atinge o `fill_rate_threshold` definido. Apenas quando estas três condições são cumpridas é gerado um alerta, garantindo que o gestor receba apenas notificações acionáveis e relevantes.

E. Limitações e Melhoramentos

Reconhecemos que o executor atual assume uma correspondência exacta nos tipos de `issues`, o que limita a flexibilidade para variações semânticas (ex: "ruptura" vs "falta de stock"). Adicionalmente, o filtro temporal (`time_filter`) é atualmente informativo, pois o sistema de inspeção não correlaciona o `timestamp` da imagem com a hora do dia de forma isolada. Como trabalho futuro, a implementação de `jsonschema` para validação rigorosa de tipos e a expansão da ontologia de `issues` são passos fundamentais para elevar a robustez da engine.

V. RAG MEMORY

O componente `rag_memory.py` implementa memória persistente que indexa *summaries* das inspeções em ChromaDB e permite recuperação semântica em linguagem natural.

A. Stack Tecnológico

- **Vector Store:** ChromaDB com `PersistentClient` (`path='./vectorstore'`). Persiste automaticamente em disco entre sessões.
- **Embeddings:** *sentence-transformers/all-MiniLM-L6-v2* (padrão ChromaDB), com suporte razoável a português.
- **LLM de Síntese:** Gemini 2.5 Flash, recebe o contexto recuperado e produz resposta com referência explícita aos IDs das inspeções.
- **Retrieval:** Similaridade de cosseno com *top-k=3* por defeito.

B. Estratégia de Chunking

Foi implementada a abordagem híbrida: o *summary* da inspeção completa é indexado como *chunk* único, com metadados estruturados (`zone_id`, `timestamp`, `overall_status`) para filtragem pré-*retrieval*.

TABLE IV
COMPARAÇÃO DE ABORDAGENS DE *chunking*

Abordagem	Vantagens	Desvantagens
Record completo(implementada)	Contexto completo; índice pequeno	Vector médio pode ser menos preciso
Por <i>issue</i>	Recuperação granular	índice cresce; contexto fragmentado
Híbrido	Balanceado; permite filtros	Mais complexo de implementar

C. Geração de Summaries

O método `generate_rich_summary` usa um *prompt* que instrui o Gemini 2.5 Flash a produzir um parágrafo denso (máximo 4 linhas) sem formatação, mencionando explicitamente: zona, estado global, *fill rate*, detalhes visuais e anomalias. Este design garante *summaries* semanticamente ricos para recuperação por zona, tipo de problema e data.

VI. AVALIAÇÃO

A fiabilidade de um sistema autónomo de retalho depende da sua capacidade de produzir diagnósticos consistentes sob condições de ruído visual. A avaliação do *Retail Vision Intelligence System* foi conduzida através de um *harness* de testes automatizado (`evaluate.py`), que permite a execução de testes *end-to-end* e a geração automática de métricas de performance.

A. Harness de Avaliação e Metodologia

O *harness* implementado automatiza a comparação entre os outputs do sistema e um *ground truth* de referência. Esta abordagem permitiu submeter as 15 imagens de teste a três estratégias de *prompting* distintas, garantindo que o ciclo de avaliação fosse reproduzível. A métrica fundamental de sucesso não foi apenas a detecção de anomalias, mas a resiliência do sistema em manter o formato estruturado (JSON) mesmo sob carga de API elevada.

B. Validação via LLM-as-a-Judge

Dado que a avaliação de "alucinação" é subjetiva, utilizámos a técnica *LLM-as-a-Judge*. Um segundo modelo Gemini (configurado como juiz) recebeu a imagem original, o `model_reasoning` gerado e a lista de *issues* detetados. O juiz foi instruído a validar a veracidade das afirmações contra o conteúdo visual. Esta técnica permitiu quantificar a *Hallucination Rate* de forma semi-automatizada, revelando que a Estratégia B (CoT) alcançou 0% de alucinações, confirmando que o raciocínio forçado é a melhor defesa contra a inventividade excessiva do modelo.

C. Avaliação da Rule Engine

A performance da `rule_engine.py` foi validada através de *input* sintético com níveis variados de ambiguidade.

TABLE V
MÉTRICAS DE PERFORMANCE DA RULE ENGINE

Métrica	Resultado Estimado
Rule Parse Rate	≈ 90%
Rule Correctness (Dados Sintéticos)	≈ 85%
Ambiguity Detection Rate	≈ 80%

Observámos que o sistema é robusto na rejeição de regras sem `zone_id` ou `thresholds`, mas a precisão na interpretação de períodos temporais (ex: "durante a tarde") permanece um desafio técnico, uma vez que o modelo tem dificuldade em mapear o tempo relativo para o estado estático da inspeção.

D. Avaliação do RAG

A eficácia do `rag_memory.py` foi testada com queries qualitativas (ex: "Qual a evolução do fill rate desta zona?"). O sistema recuperou documentos relevantes em 80% das tentativas (Recall@3). Em todas as respostas geradas, o modelo manteve uma *Faithfulness* de 100%, referenciando explicitamente o `inspection_id` e o `timestamp` dos eventos, o que é crucial para garantir confiança ao utilizador final (gestor de loja).

E. Limitações e Discussão

Durante a avaliação, enfrentámos desafios significativos de infraestrutura:

- **Erros de API (503/429):** A dependência da API gratuita do Gemini Flash expôs limitações de *throughput*. A mitigação ocorreu via persistência em *cache* local com *hashing* MD5, permitindo que cada teste fosse executado apenas uma vez contra o servidor.
- **Viés do Ground Truth:** O *ground truth* original foi definido com um único *issue* por imagem. Contudo, o sistema, sendo minucioso, detetava múltiplos problemas reais. Isto inflacionou artificialmente a *False Positive Rate* (até 80% em algumas estratégias), uma vez que o sistema foi penalizado por encontrar problemas que não estavam na anotação manual.
- **Iluminação e Ruído:** As imagens com blocos pretos de privacidade (técnica de anonimização) degradaram severamente a *Severity Accuracy*, demonstrando que o modelo é sensível a obstruções artificiais que não são típicas da sua distribuição de treino.

VII. INTEGRAÇÃO COM PROJECTO 1

A integração com os dados de trajetória do Projeto 1 não foi implementada nesta versão. A arquitetura foi, no entanto, desenhada para suportá-la: o `report_generator.py` inclui uma secção dedicada que apresenta uma nota informativa quando os dados não estão disponíveis.

A integração potencial funcionaria da seguinte forma: os dados de afluência por zona e período (do Projeto 1) seriam incluídos nos metadados das inspeções no ChromaDB. Uma inspeção que detectasse prateleira vazia na zona Z_S1 às 15h poderia ser contextualizada com afluência 40% acima da média, distinguindo entre falha de reposição e esgotamento por procura elevada.

VIII. CONCLUSÃO

A. O que funcionou

A Estratégia B (CoT) foi a mais equilibrada: 0% de alucinações, FPR razoável e raciocínio completamente verificável no `model_reasoning`. Para aplicações de produção onde fiabilidade supera *recall*, o CoT é a escolha clara. O sistema de *cache* por MD5 funcionou perfeitamente, permitindo múltiplas iterações de desenvolvimento sem consumo adicional de quota. O *Rule Engine* funcionou bem para regras acionáveis, com detecção robusta de ambiguidades.

B. O que não funcionou

A Estratégia A tem *suggestion bias* severo: as instruções específicas sobre produtos inexistentes nas imagens testadas

causaram 42,9% de alucinações. O Issue Detection Rate de todas as estratégias ficou abaixo de 40%, refletindo tanto limitações do *ground truth* como dificuldades reais na detecção de problemas subtis de planograma. A Severity Accuracy da Estratégia B (25%) é baixa — o CoT produz raciocínio correcto mas calibra mal a severidade.

C. Trabalho Futuro

Três melhorias principais são propostos:

(1) **Estratégia D = CoT + Few-Shot**, combinando a eliminação de alucinações do CoT com exemplos calibrados para severidade;

(2) **Ground truth granular** com múltiplos *issues* por imagem e cálculo de IoU de *bounding boxes*;

(3) **Filtragem temporal explícita** no RAG, extraíndo hora do dia dos *timestamps* ISO 8601 para suportar *queries* temporais.

Link para o repositório no github: https://github.com/santosines/tp2_intera-ao_ines.git